Super-Resolution GAN for Dog/Cat Image Classification

Applied AI — Midterm Exam Report

DSBA 6165 – AI & Deep Learning

Rony Reyes (rreyespi)

1 Introduction

The project is about to combine two ideas from deep learning: **image classification** (deciding whether a picture shows a dog or a cat) and **image super-resolution** (taking a small, blurry picture and generating a bigger, more detailed version of it). The tool used for super-resolution is a **Generative Adversarial Network (GAN)**, specifically a design called **SRGAN** (Super-Resolution GAN).

A GAN is made of two neural networks that compete with each other during training: a **generator**, which tries to produce realistic-looking images, and a **discriminator**, which tries to tell the difference between real images and the generator's fakes. As training goes on, the generator gets better at fooling the discriminator, and the discriminator gets better at catching fakes where each pushes the other to improve.

Exam's requirements:

- A classifier (**Model A**) trained the normal way, using real 128×128 images.
- An SRGAN trained to turn 32×32 low-resolution images into 128×128 high-resolution images, for at least 150 training epochs.
- A second classifier (**Model B**) trained using the SRGAN's generated images instead of real ones.
- A comparison of Model A and Model B using Accuracy, F1 score, and AUC.
- A 70% / 30% train/test split, normalization, and example images at each stage.

Refer to file **Midterm_Exam_rreyespi_local.ipynb** for a full implementation of this exam in python.

How Model A connects to the SRGAN

Model A itself is a fine-tuned **VGG16** network. VGG16 is a well-known convolutional neural network (CNN) originally trained on millions of general photographs (the ImageNet dataset). **Transfer learning** means we reuse VGG16's already-learned ability to recognize shapes, textures, and edges, and then re-train (fine-tune) just the last part of it for our specific dog-vs-cat task, which is much faster than training a whole CNN from scratch.

This project reused Model A in a second way, beyond just being a classifier to compare against: its fine-tuned VGG16 layers were also used to help train the SRGAN's generator. Instead of judging the generator's output only by raw pixel differences, we compared *feature maps* extracted from an inner layer of Model A's VGG16 between the real image and the generated image. This is called a **perceptual loss** and the idea is that matching what a trained classifier "notices" about an image is a better training signal than matching pixels directly, since it pushes the generator toward reproducing the visual patterns that actually matter for telling dogs and cats apart.

2 Dataset, Split, and Preprocessing

The dataset consists of labeled photographs of cats and dogs. Initially, all images were combined and then randomly split into **70% for training and 30% for testing**, keeping the same proportion of cats and dogs in both sets (a *stratified* split).

Every image was prepared in two versions:

- **HR (high-resolution)**: resized to 128×128 pixels — this is the size used by both classifiers, and the target size the SRGAN tries to reconstruct.
- **LR (low-resolution)**: the HR image shrunk further down to 32×32 pixels — this simulates a genuinely blurry, low-detail photo, and is what the SRGAN's generator receives as input.

Normalization means rescaling pixel values into a consistent numeric range before feeding them to a neural network, which helps training converge more smoothly. Images fed into the GAN were scaled to the range $[-1, 1]$ (matching the generator's final activation function, *tanh*), while images fed into the VGG16-based classifiers used the standard preprocessing VGG16 expects.

Figure 1: Five sample images at each stage: the original photo, the 128×128 HR version, and the 32×32 LR version. Notice how much visual detail is lost once an image is shrunk to only 32×32 pixels — recovering that detail is the SRGAN's goal.

3 Model A — Classifier Trained on Real Images

Model A was trained in two phases, a standard transfer-learning approach:

- **Phase 1 — Feature extraction:** VGG16's convolutional layers were frozen (not updated), and only a small classification **head** added on top (a pooling layer, a Dense layer, Dropout for regularization, and a final Dense layer with a sigmoid output for the cat/dog decision) was trained.
- **Phase 2 — Fine-tuning:** the last convolutional block of VGG16 (block5) was unfrozen and trained too, using a much smaller learning rate so the already-useful VGG16 weights aren't disrupted too aggressively.

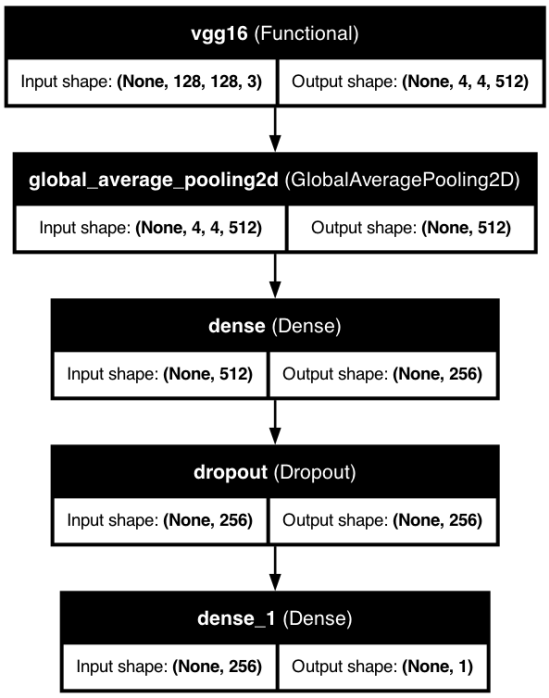

Figure 2: Model A's architecture: the frozen/fine-tuned VGG16 base, followed by global average pooling and a small classification head.

Layer (type)	Output Shape	Param #
vgg16 (Functional)	(None, 4, 4, 512)	14,714,688
global_average_pooling2d (GlobalAveragePooling2D)	(None, 512)	0
dense (Dense)	(None, 256)	131,328
dropout (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 1)	257

Table 1: Model A's layer summary, from its own `model.summary()` output.

Which layers actually get fine-tuned

The table below lists every layer in the VGG16 base and whether it was trainable during Phase 2 (fine-tuning). Only the last convolutional block, block5, was unfrozen; everything earlier in the network

(which tends to capture more generic, reusable visual features like edges and textures) stayed frozen, while block5 (which captures more task-specific, higher-level features) was allowed to adapt to the dog/cat task.

Layer	Trainable
input_layer	False
block1_conv1	False
block1_conv2	False
block1_pool	False
block2_conv1	False
block2_conv2	False
block2_pool	False
block3_conv1	False
block3_conv2	False
block3_conv3	False
block3_pool	False
block4_conv1	False
block4_conv2	False
block4_conv3	False
block4_pool	False
block5_conv1	True
block5_conv2	True
block5_conv3	True
block5_pool	True

Table 2: Layer-by-layer trainability of VGG16 during Phase 2 fine-tuning (green rows = trainable, matching block5).

Training curves

The chart below shows training and validation accuracy/loss across both phases. **Accuracy** is the fraction of images classified correctly; **loss** is the number the optimizer is directly trying to minimize (lower is better). A model that's learning well shows accuracy going up and loss going down over time, with training and validation curves staying reasonably close together (a big gap between them would suggest *overfitting* — memorizing the training images rather than learning general patterns).

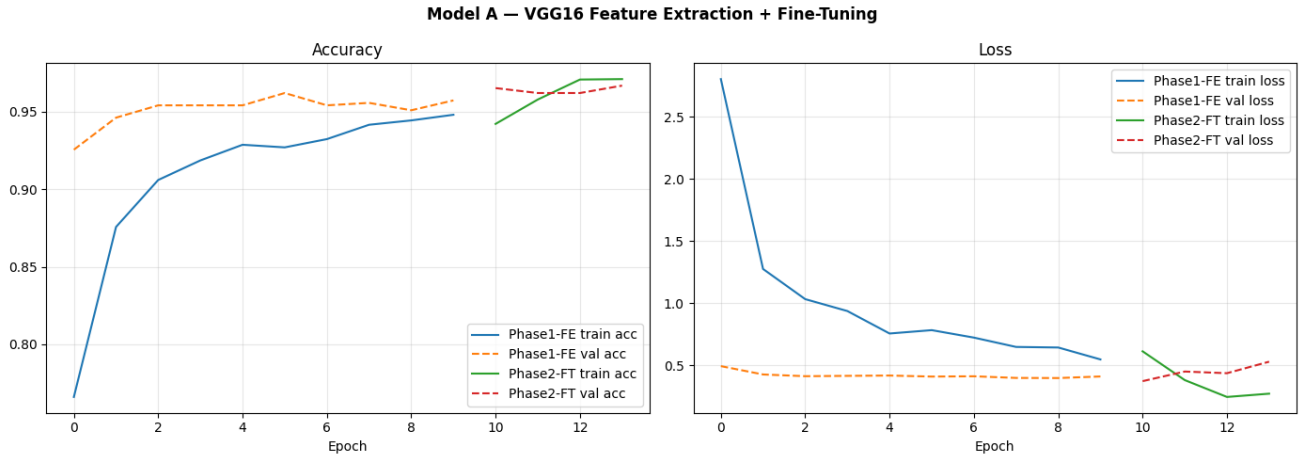

Figure 3: Model A's accuracy and loss during Phase 1 (feature extraction, epochs 0–9) and Phase 2 (fine-tuning, epochs 10 onward). Validation accuracy reaches roughly 96–97%, and fine-tuning gives a further small improvement over Phase 1 alone.

4 The Super-Resolution GAN

As introduced above, the SRGAN has two networks trained together: a **generator** that upscales a 32×32 image to 128×128, and a **discriminator** that tries to spot which 128×128 images are real photos versus generator output.

The role of the Generator

The generator is the network doing the actual upscaling work: it takes a small, blurry 32×32 image and tries to produce a convincing 128×128 version of it. Nothing about the generator knows whether its own output looks real; it only improves because it's graded on three things every training step: how close its output's raw pixels are to the real image (pixel MSE), how close its output's learned features are to the real image (the perceptual/content loss, using Model A's fine-tuned VGG16), and whether it can fool the discriminator (the adversarial loss).

The role of the Discriminator

The discriminator is a classifier too, but not a dog-vs-cat classifier; it's a **real-vs-fake** classifier. It's shown a mix of real HR photos and the generator's output, and its only job is to output a probability that a given image is real. It's trained the same way any binary classifier is trained (real images should score close to 1, generated images close to 0). Its purpose isn't to be useful on its own; it exists purely to give the generator a harder, more realistic training signal than pixel-matching alone would provide. As the generator improves, the discriminator has to get better at spotting subtler fakes, which in turn pushes the generator to improve further — that back-and-forth competition is the "adversarial" part of GAN.

One subtlety worth calling out here, since it shows up again in the training curves in Section 5: the generator and discriminator are trained on *different* loss functions computed from overlapping information. The discriminator's own loss only cares about correctly labeling real vs. fake. The generator's loss borrows the discriminator's real/fake score as just one ingredient (weighted very lightly, ×0.001) alongside the much larger-scale perceptual and pixel losses.

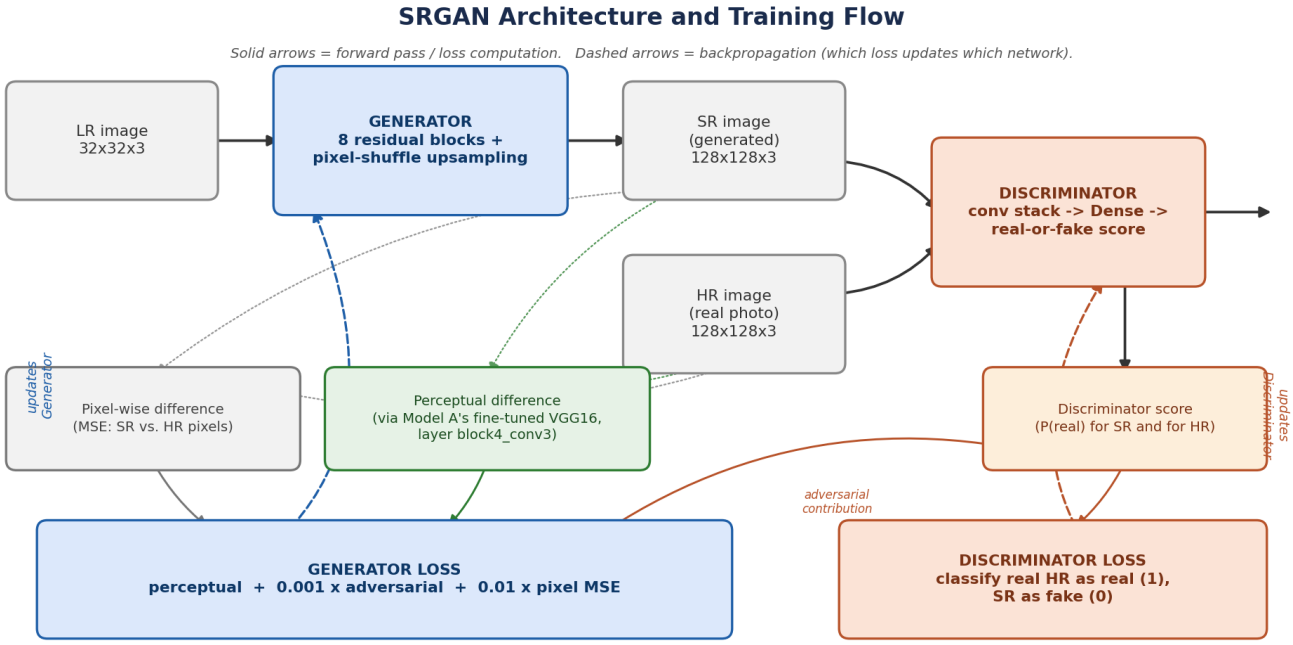

Figure 4: Overview of the SRGAN architecture and training flow. Solid arrows show the forward pass and loss computation; dashed arrows show which loss updates which network during backpropagation.

Generator architecture, by stage

The generator takes the small 32×32 image and passes it through a stack of **residual blocks** (eight of them, in this design). A residual block lets the network learn a small correction to add to its input rather than having to reconstruct the entire signal from scratch at every layer; this is a common trick (popularized by the ResNet architecture) that makes deep networks easier to train. After the residual blocks, the image is enlarged using two "pixel-shuffle" upsampling steps, each doubling the width and height (32→64→128), before a final convolution produces the 3-channel (RGB) output image.

Stage	What it does	Output size
Input	Low-resolution image	32 x 32 x 3
Initial conv + activation	Extracts first set of features	32 x 32 x 64
8 x Residual blocks	Refine features while preserving detail	32 x 32 x 64
Upsample block 1	Pixel-shuffle upscaling (2x)	64 x 64 x 64
Upsample block 2	Pixel-shuffle upscaling (2x)	128 x 128 x 64
Final conv (tanh)	Produces the RGB super-resolved image	128 x 128 x 3

Table 3: Generator architecture, summarized by stage.

Discriminator architecture, by stage

The discriminator is a more conventional CNN classifier: a sequence of convolutional layers that repeatedly shrink the image's spatial size while increasing the number of feature channels (64 → 512), followed by a large Dense layer and a final sigmoid output that gives a single real-vs-fake probability.

Stage	What it does	Output size
Input	Real HR image or generator output	128 x 128 x 3
4 conv blocks (stride 1, then 2)	64 -> 64 channels, halves spatial size once	64 x 64 x 64
4 more conv blocks (stride 1, then 2)	128 -> 128 -> 256 -> 256 channels	16 x 16 x 256
2 more conv blocks (stride 1, then 2)	512 -> 512 channels	8 x 8 x 512
Flatten + Dense(1024)	Combines all spatial features	1024
Dense(1, sigmoid)	Real-vs-fake probability	1

Table 4: Discriminator architecture, summarized by stage.

How the generator and discriminator are trained together

At each training step, the generator produces a batch of super-resolved images from LR inputs, and the discriminator scores both those and a batch of real HR images. The discriminator is trained to give real images a high real score and generated images a low one. The generator, meanwhile, is trained using three combined signals: the **perceptual loss** described above, an **adversarial loss** that rewards the

generator whenever it fools the discriminator, and a small **pixel-wise loss** (plain mean-squared-error against the real HR image) that helps keep training stable early on.

5 SRGAN Training Results

The SRGAN was trained for 150 epochs (an **epoch** is one full pass through the training data), which was the minimum required by the exam. Several things were tracked during training to evaluate how well it worked.

Loss curves

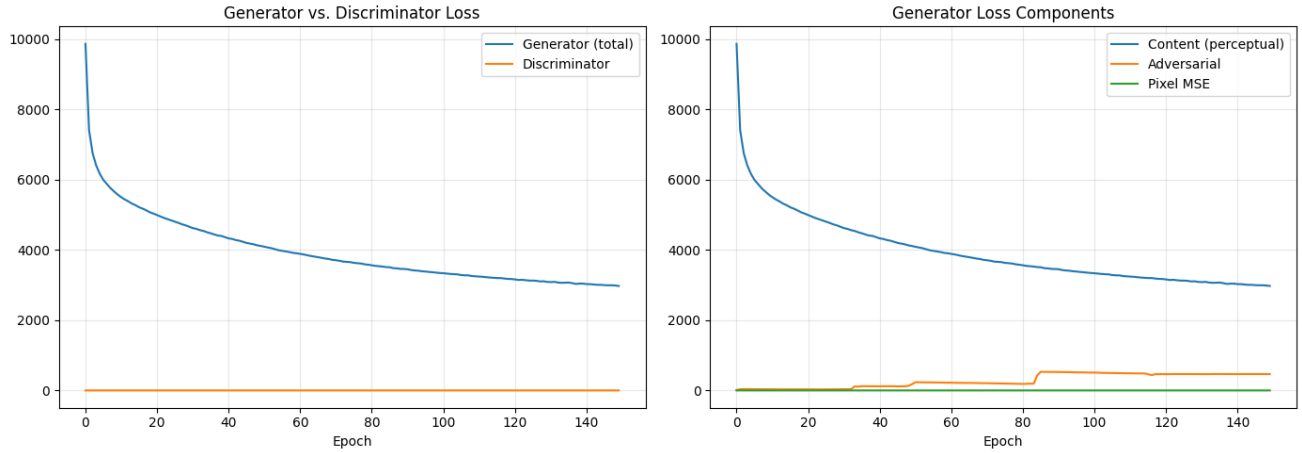

Figure 5: Left: total generator loss vs. discriminator loss. Right: the generator's three loss components plotted separately.

One thing worth pointing out as a learning example: the discriminator loss (orange line, left chart) looks like it's sitting flat at zero, but it isn't actually zero — it's just tiny compared to the perceptual/content loss, which starts around 10,000 and is plotted on the same scale. This is a common pitfall when plotting several loss terms together: very different scales can hide real changes in the smaller-scale curve. The content loss (large scale, blue line in the right-hand chart) dominates the total generator loss and drives most of its visible improvement, dropping from about 10,000 to about 3,000 over training, while the adversarial and pixel-MSE terms (much smaller in scale) stay relatively flat by comparison.

PSNR and SSIM: comparing against a simple baseline

Two standard metrics for judging super-resolution quality are **PSNR** (Peak Signal-to-Noise Ratio, measured in decibels — higher means the reconstructed image is closer pixel-for-pixel to the real one) and **SSIM** (Structural Similarity, a score between 0 and 1 that tries to capture perceived structural similarity rather than raw pixel differences). To judge whether the SRGAN was actually adding value, both metrics were also computed for a much simpler baseline: just resizing the 32×32 image straight up to 128×128 using ordinary bicubic interpolation, with no learning involved at all.

This is a genuinely useful (if humbling) result to discuss, not a failure to hide. It's actually a well-known pattern with GAN-based super-resolution: PSNR and SSIM specifically reward matching the exact pixel values of the ground-truth image, but the adversarial and perceptual losses are explicitly pushing the generator toward images that *look* sharp and detailed to a feature extractor or a discriminator, even if that means adding fine texture that doesn't exactly match the original pixels. In other words, this SRGAN was trained to optimize for something other than PSNR/SSIM, so it's not surprising it doesn't

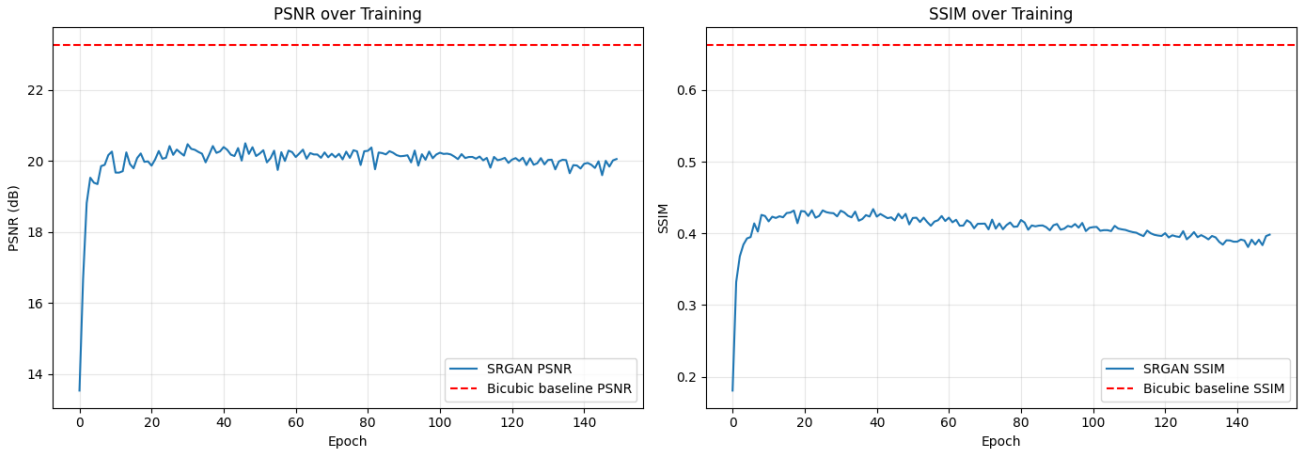

Figure 6: SRGAN's PSNR and SSIM across training (blue), compared against the no-learning bicubic baseline (dashed red line).

win on those particular metrics. The qualitative image comparisons below help show what that trade-off actually looks like.

Qualitative results: what the images actually look like

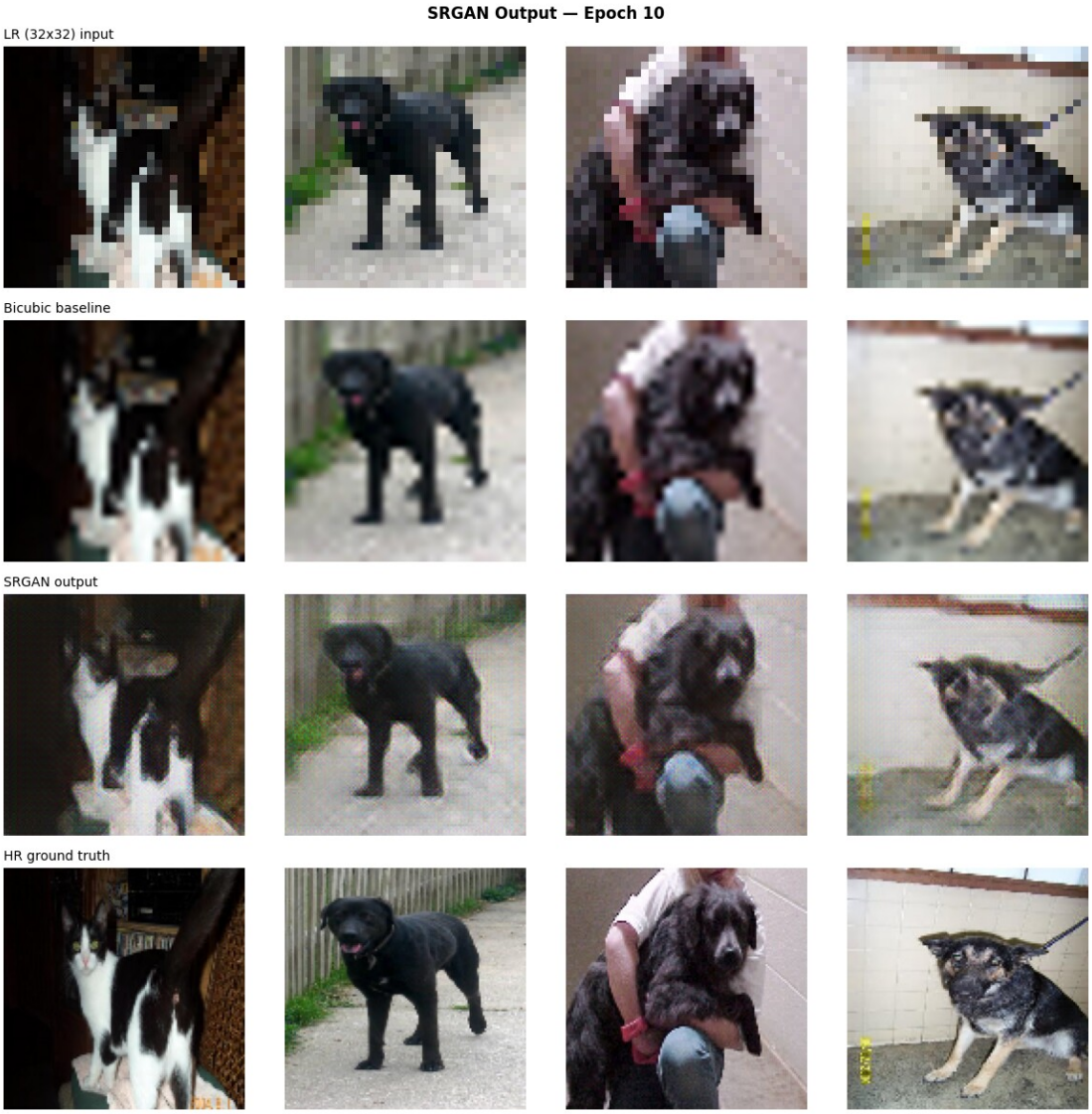

Figure 7: Comparison after only 10 training epochs: LR input, bicubic baseline, SRGAN output, and the real HR ground truth.

Figure 8: The same comparison at the end of training (epoch 150), on a fresh set of test images.

Looking at these side by side, the bicubic baseline produces a soft, blurry but pixel-accurate image, while the SRGAN output is visibly sharper in places but has a distinctive fine speckled/noisy texture across the whole image, even in flat areas like backgrounds. That speckling is consistent with the lower PSNR/SSIM numbers above: the GAN is adding detail, but not always the *correct* detail, which trades pixel accuracy for a sharper-looking (if noisier) result.

Pixel-error distribution

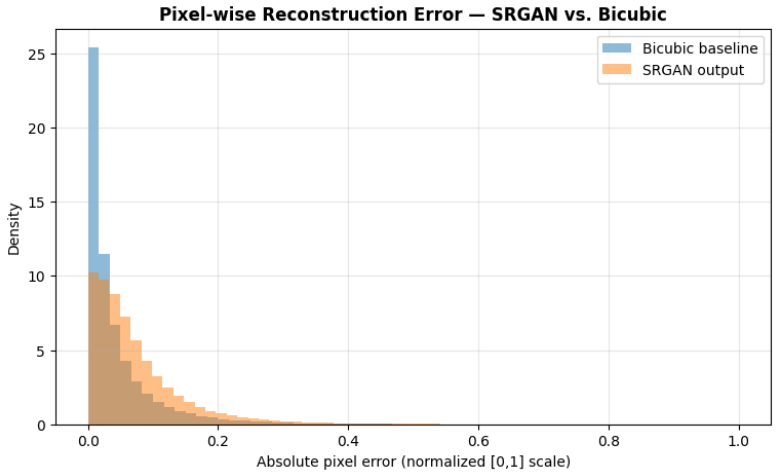

Figure 9: Distribution of absolute per-pixel error against the real HR image, for the bicubic baseline vs. the SRGAN. The bicubic baseline's errors are concentrated much closer to zero, visually confirming the PSNR/SSIM gap above.

6 Model B — Classifier Trained on SRGAN Images

Model B uses the exact same architecture and training recipe as Model A (VGG16 feature extraction, then fine-tuning). The only difference is what it was trained on: instead of real 128×128 photos, every training image's 32×32 LR version was passed through the trained SRGAN generator, and Model B was trained on those generated images instead.

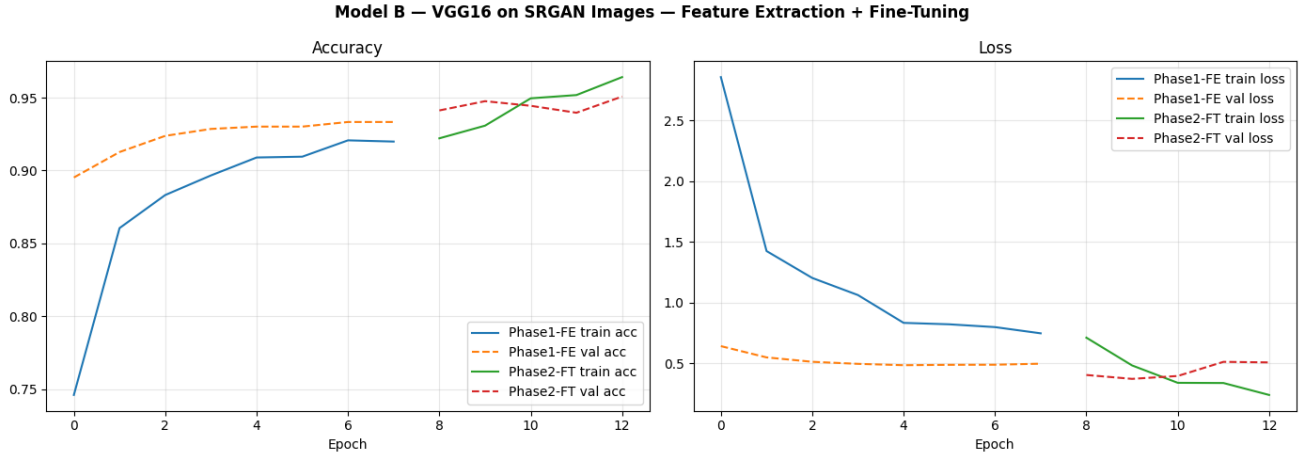

Figure 10: Model B's accuracy and loss curves. Despite training on the SRGAN-generated (lower PSNR/SSIM) images, Model B's curves look very similar in shape to Model A's, reaching a comparable validation accuracy.

7 Comparing Model A vs. Model B

Both models were evaluated on the same real-image test set (the 30% held out earlier), so the comparison below isolates the effect of what each model was *trained* on.

Metric	Model A (real HR images)	Model B (SRGAN images)
Accuracy	95.9%	96.1%
F1 score (dog class)	0.959	0.960
AUC	0.992	0.993

Table 5: Test-set metrics, computed directly from the confusion matrices in Figure 12 (exact values); AUC values read from Figure 13's legend.

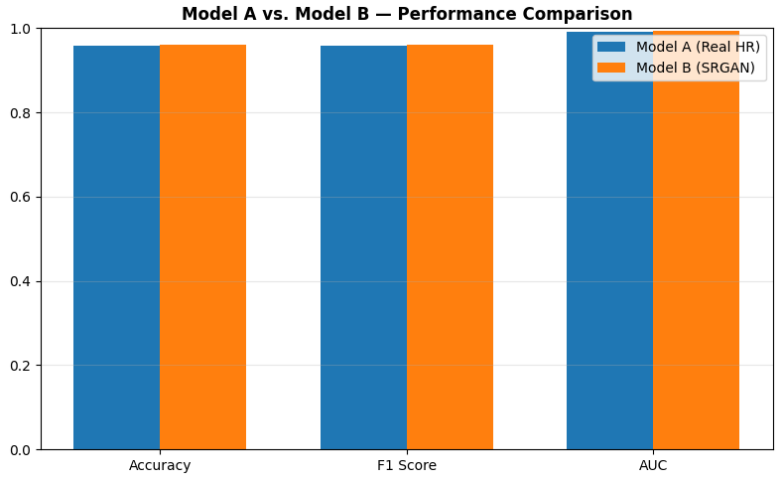

Figure 11: Side-by-side bar chart of the same three metrics.

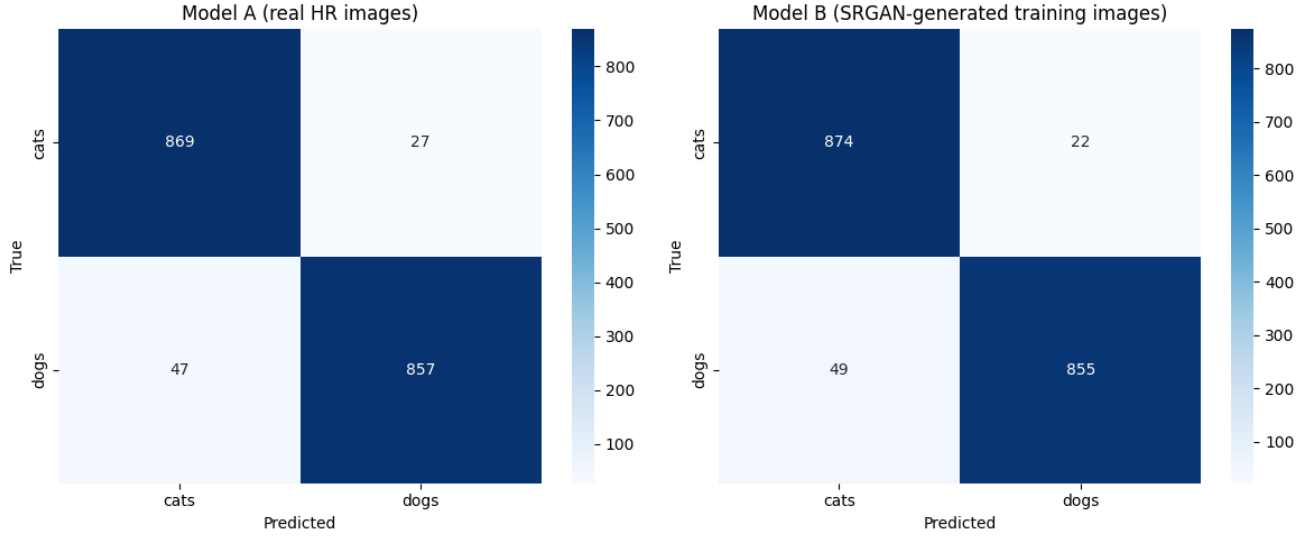

Figure 12: Confusion matrices on the shared real-image test set (1,800 images total). Model A: 869 cats and 857 dogs correctly classified. Model B: 874 cats and 855 dogs correctly classified.

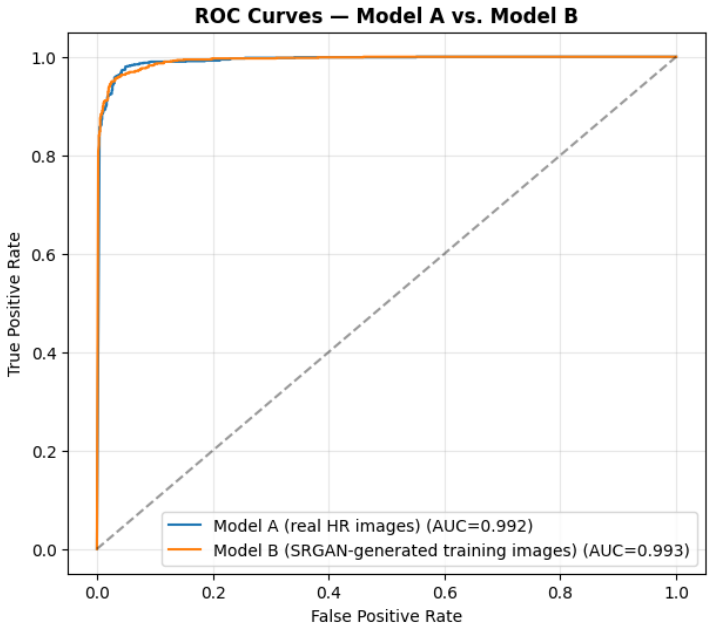

Figure 13: ROC curves for both models. An ROC curve plots how well a classifier trades off catching true positives against raising false alarms as its decision threshold changes; AUC (area under this curve) summarizes that trade-off in a single number from 0.5 (no better than guessing) to 1.0 (perfect).

8 Discussion

To my surprise, Model B, trained entirely on SRGAN-generated images with visibly lower PSNR/SSIM than a plain bicubic upscale, performed essentially the same as Model A, which was trained on real photos. The differences (96.1% vs. 95.9% accuracy, 0.993 vs. 0.992 AUC) are small enough that they could just be run-to-run noise rather than a real effect, but the key takeaway either way is that the SRGAN's speckled, imperfect reconstructions *did not meaningfully hurt classification performance.*

A few limitations of this specific analysis, worth naming directly:

- Only a subset of the full dataset was used (to keep training time reasonable), so these results should be treated as a proof-of-concept rather than a definitive benchmark.
- 150 epochs is the exam's minimum requirement, but the PSNR/SSIM curves in Figure 6 look fairly flat well before epoch 150, suggesting more epochs likely wouldn't have changed the SRGAN's plateaued performance much without other changes (e.g., loss re-weighting, a different learning rate schedule, or a larger dataset).
- The very large gap in scale between the content loss and the adversarial/discriminator loss (Figure 5) means the adversarial signal may be contributing very little to training in practice — re-weighting those loss terms could be a natural next experiment.

9 Conclusion

This project built a working SRGAN that upsamples 32×32 images to 128×128, reused a fine-tuned VGG16 classifier's features to help train it, and used its output to train a second classifier. The SRGAN's reconstructions score below a simple bicubic resize on standard pixel-fidelity metrics (PSNR/SSIM),

which is a known trade-off with adversarially-trained super-resolution models. Despite that, a classifier trained entirely on the SRGAN's output matched the performance of one trained on real images, which suggests that even an imperfect SRGAN can preserve the information a downstream classifier actually needs.

Appendix: Configuration Used

Setting	Value
HR image size	128 x 128
LR image size	32 x 32 (4x upscale factor)
Train / test split	70% / 30%, stratified by class
Classifier batch size	32
SRGAN batch size	16
SRGAN training epochs	150
Checkpoint frequency	Every 10 epochs
Classifier backbone	VGG16 (ImageNet-pretrained), fine-tuned
SRGAN content-loss source	Model A's fine-tuned VGG16, layer block4_conv3

Table 6: Notebook configuration values assumed for this report.